2.4. Experiment 4 - Application Communication Strategy

Requirements: Compare request/response polling with publish/subscribe forwarding through a gateway. Evaluate event latency, traffic cost versus polling interval, and behavior under higher background traffic.

Procedure: Four smoke/environment sensors produce a new readout every 10 s. Eight other sensors generate background reports. The gateway connects the IEEE 802.15.4 access network to the Ethernet-side monitoring application. One actuator is retained. Node positions and radio settings are fixed across the comparison.

Setting	Value
Duration / seeds	1800 s / seed sets 0-4
Sensor event schedule	1 + 0.03 x sensor index, then every 10 s
Event count	180 per sensor; 720 total per run
Polling intervals, T	1, 5, 10, 30, 60 s
Background intervals, H	10 s (normal) and 0.1 s (high), per background sensor
Offered background load	0.8 versus 80 packets/s; 50-byte payloads
Request / response / publication size	20 / 50 / 50 bytes of application payload
Total runs	50 request/response + 10 publish/subscribe = 60

Strategy A - Request/response: The core application runs one poller per sensor. A request passes through the gateway to the sensor, which returns its latest readout through the gateway. Each poller starts at 1.2 s plus a uniform random phase over its polling interval. Repeated replies carrying the same sensor value count only once for event coverage.

Strategy B - Publish/subscribe: Each sensor publishes its new readout to a UDP broker application at the gateway. The broker forwards one notification to the statically configured core subscriber. Sensor identity and original event time are retained across forwarding.

Notes: These are behavioral CoAP-like and MQTT-like strategies over UDP, not full implementations of those protocols. There is no TCP, application retry, QoS negotiation or subscription handshake. Sensor value changes are periodic; the polling sensor computes the latest value from the same schedule used by publishing. Five seeds vary random start times and wireless backoff.

Source/configuration: src/StrategyApp.cc, src/StrategyApp.ned and simulations/experiment4.ini. The implementation reuses INET 4.6 UdpBasicApp socket/lifecycle behavior and the existing SmartHomeNetwork routing.

Experiment 4 - Results

Each row averages five 1800-second runs, except maximum event latency, which is the largest observed value. Event latency includes waiting for the core to learn a changed sensor value, while polling RTT measures only a request/reply exchange.

Load	Strategy	Event coverage (%)	Mean event latency (ms)	Max event latency (ms)	Transaction delivery (%)	App messages / run
Normal	Poll 1s	100.00	586.30	1012.71	99.992	14391.2
Normal	Poll 5s	100.00	2523.92	4465.64	100.000	2879.6
Normal	Poll 10s	100.00	4884.43	8780.60	100.000	1440.0
Normal	Poll 30s	33.33	4826.35	9624.67	100.000	480.0
Normal	Poll 60s	16.67	3989.25	9099.32	100.000	240.0
Normal	Publish	100.00	4.70	24.81	100.000	1440.0
High	Poll 1s	100.00	597.68	1813.40	99.497	14376.2
High	Poll 5s	100.00	2544.95	9496.23	99.722	2878.4
High	Poll 10s	99.89	4889.59	8804.38	99.889	1439.4
High	Poll 30s	33.22	4828.85	9645.00	99.667	479.6
High	Poll 60s	16.61	3992.68	9143.86	99.667	239.8
High	Publish	99.69	7.48	62.11	99.694	1437.8

Table 4.1. Event coverage = unique received events / 720. Transaction delivery = replies / requests for polling, or notifications / publications for publishing. Application messages exclude ordinary background traffic and do not count IP routing as an extra application send.

Experiment 4 - Traffic and Energy Cost

Load	Strategy	Poll RTT (ms)	App payload bytes / run	Broker forwards / run	Network energy (J)
Normal	Poll 1s	8.985	503686	0.0	59.727
Normal	Poll 5s	8.438	100786	0.0	53.015
Normal	Poll 10s	8.459	50400	0.0	52.150
Normal	Poll 30s	8.435	16800	0.0	51.440
Normal	Poll 60s	8.425	8400	0.0	51.273
Normal	Publish	-	72000	720.0	51.615
High	Poll 1s	14.309	502936	0.0	164.272
High	Poll 5s	12.811	100726	0.0	156.533
High	Poll 10s	12.576	50370	0.0	155.615
High	Poll 30s	12.026	16780	0.0	155.041
High	Poll 60s	11.795	8390	0.0	154.875
High	Publish	-	71890	717.8	155.214

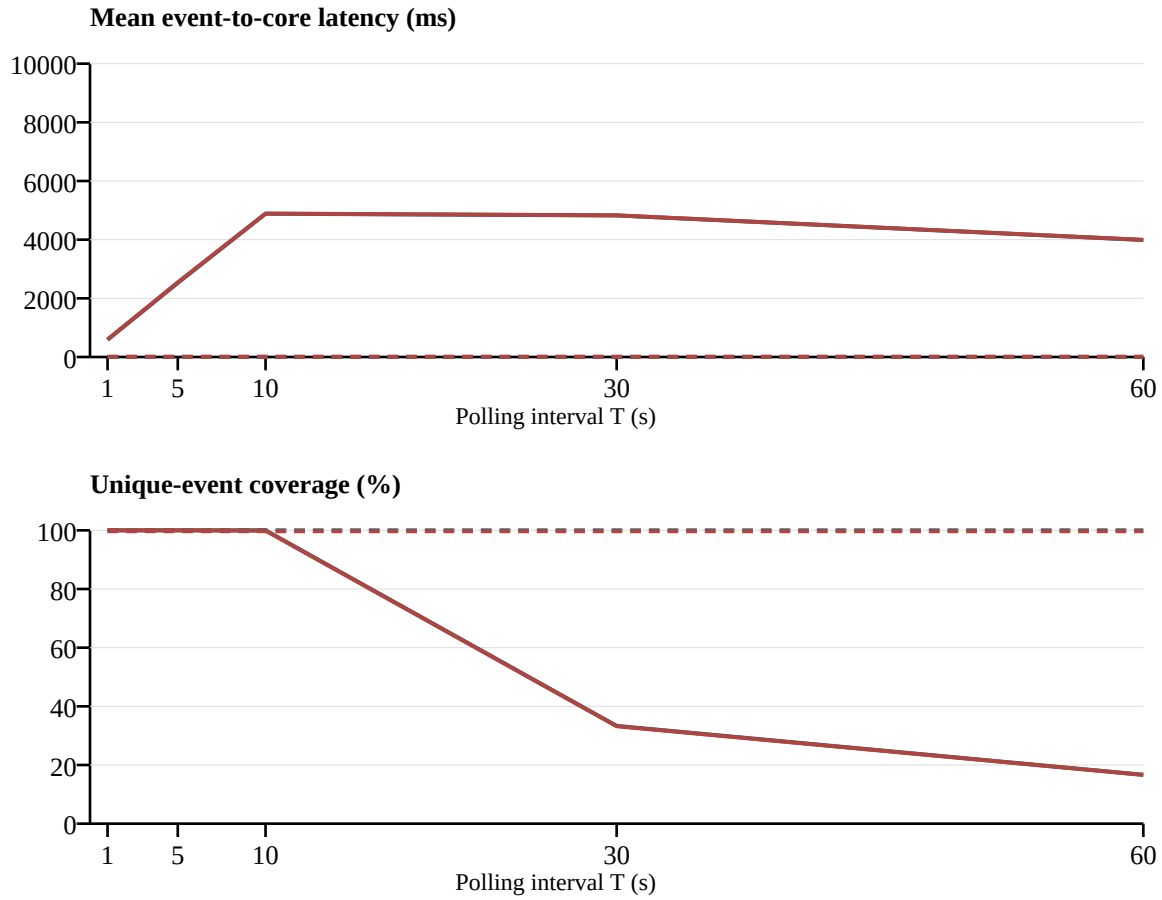
Table 4.2. Application payload cost is 20 bytes per poll plus 50 bytes per response/publication/notification. Each broker forward adds 50 application bytes. UDP/IP/MAC overhead and physical retransmissions are excluded. Energy sums initial minus final node storage under the inherited energy model.

Normal and high loads use the same eight background sources. The high setting offers 100 times the background reporting rate. Traffic shown here counts only the strategy applications, while the network energy includes background traffic and all modeled node storage. These quantities therefore have different scopes.

Polling repeats request/reply exchanges even if a sensor value has not changed. Publishing sends one update and one broker notification per change, so its cost tracks the event generation rate. Slow polling reduces traffic further, but the coverage results show how many intermediate values it misses.

Experiment 4 - Comparison and Discussion

Blue: normal background load. Red: high background load. Solid lines: polling. Dashed lines: publish/subscribe, shown as a fixed reference because its event interval remains 10 s.



Figures 4.1-4.2. Latency must be interpreted with coverage: a low conditional delay does not account for updates that polling never observes. Overlapping curves indicate similar results at the two loads.

Publishing achieved mean event latency of 4.70 ms under normal load and 7.48 ms under high load. Normal-load one-second polling averaged 586.30 ms, because a new sensor value can wait for the next poll before its request/reply exchange begins.

At normal load, one-second polling used 14391 application messages per run versus 1440 for publishing. Polling at 10 s uses approximately the same message count as publishing but delays discovery. Slower 30 s and 60 s polling reduces traffic by skipping intermediate values: normal-load coverage is 33.33% and 16.67%, respectively.

The higher background load tests contention with the same physical layout. Transaction delivery separates communication failures from missed values caused by polling frequency. Compare both loads in Table 4.1 rather than treating every unobserved event as a dropped packet.

Experiment 4 - Reproducibility and Limitations

Reproduce: Build the project, then run RequestResponse runs 0..49 and PublishSubscribe runs 0..9 using experiment4.ini with Cmdenv. In the IDE, the working directory is /IOTProject/simulations and the executable is /IOTProject/src/smarthome. Open exp4-analysis.anf after refreshing the project.

The provided run_exp4.sh executes either configuration from the bundled OMNeT++ terminal. analyze_exp4.py verifies the complete run matrix and exports exp4_per_run.csv, exp4_summary.csv and exp4_all_scalars.csv. report_exp4.py regenerates this report and its two standalone plots. Detailed commands and metric definitions are in EXPERIMENT4.md.

Browse Data: Select the RequestResponse or PublishSubscribe experiment. At monitoringApp.app[*], uniqueEventsObserved gives distinct readouts received. For the latency screenshots, export_exp4_latency.py exposes the original statistics as ordinary scalars eventLatencyMean and eventLatencyMax in exp4-latency.anf. This derived view does not rerun the simulation or change measurements. At gateway.app[1] in PublishSubscribe, packetSent:count gives broker notifications.

Validation: All 60 parameter/seed combinations must be present. The analyzer checks 1800 s duration, seed metadata, 720 publications per publishing run, unique-event counts, nonnegative delays, and counts consistent with the request/reply or publication/forwarding chain. Short smoke tests exercise both routes before the full runs.

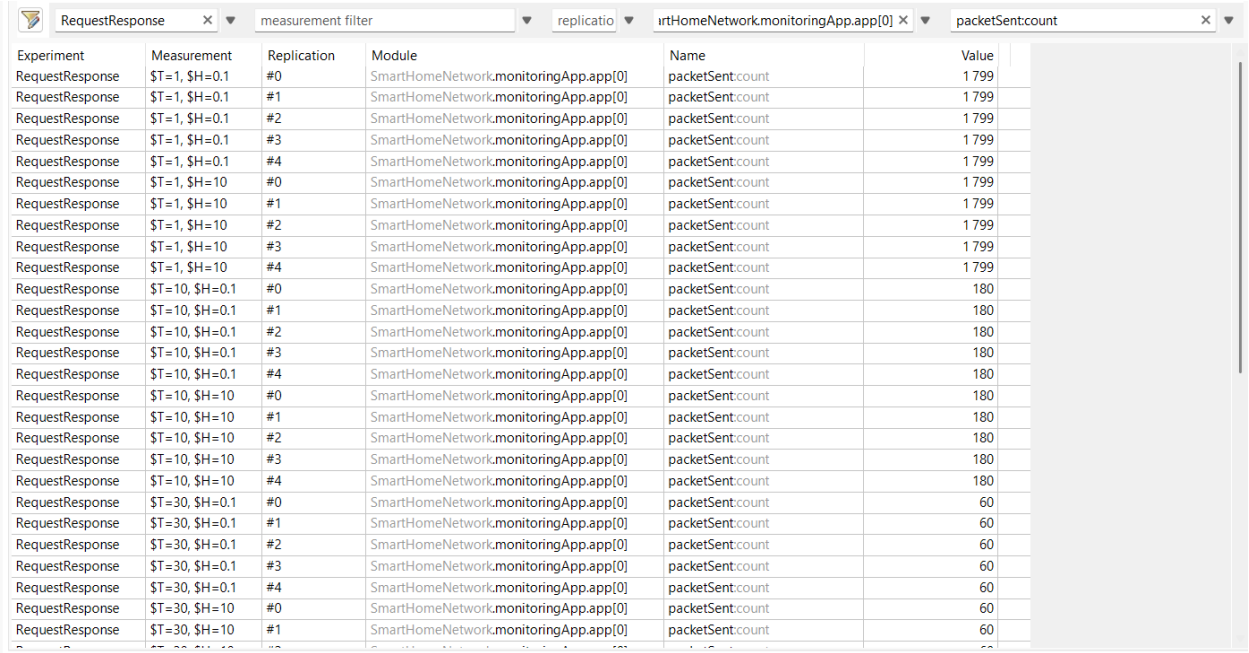
Interpretation limits: Event latency is calculated only for received distinct events. Intermediate readouts overwritten before a poll cannot be recovered. No extra drain period is added at 1800 s. Periodic event generation can phase-lock with polling intervals; five random polling phases reduce dependence on one alignment but do not establish behavior for arbitrary event arrival processes.

The static subscription excludes setup costs and assumes one subscriber. Poll requests cross a wireless hop in each direction, whereas publications cross the wireless link once and are forwarded over Ethernet. The reported application traffic metric counts application sends, not every link transmission. These results support this architecture and workload, not a universal CoAP-versus-MQTT performance ranking.

The original high-power radio settings and simplified energy model are inherited from the baseline. Energy values describe this simulation; hardware battery-life claims require calibrated device parameters. The auxiliary deliveryDelay statistic measures the last application leg and must not replace eventLatency in the strategy comparison.

Source references: Course handout, Experiment 4 (Project_22_8_2026.pdf, page 2). INET 4.6 UdpBasicApp and UdpEchoApp source in the installed INET framework. OMNeT++ 6.2.0 simulation environment. All values in this report come from the local results_strategy_1800s scalar files.

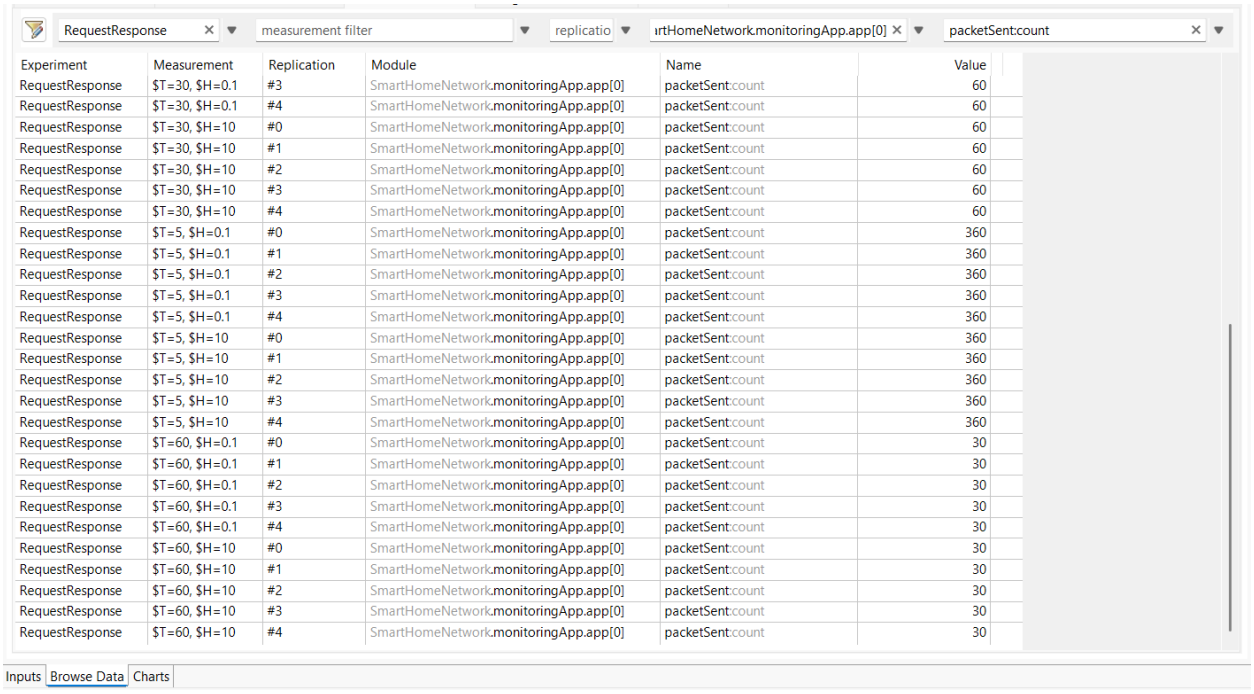
Experiment 4 - Polling Request Screenshots



RequestResponse | measurement filter | replicatio | ırtHomeNetwork.monitoringApp.app[0] | packetSent:count

Experiment	Measurement	Replication	Module	Name	Value
RequestResponse	$T=1, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	1 799
RequestResponse	$T=1, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	1 799
RequestResponse	$T=1, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	1 799
RequestResponse	$T=1, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	1 799
RequestResponse	$T=1, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	1 799
RequestResponse	$T=1, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	1 799
RequestResponse	$T=1, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	1 799
RequestResponse	$T=1, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	1 799
RequestResponse	$T=1, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	1 799
RequestResponse	$T=1, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	1 799
RequestResponse	$T=10, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	180
RequestResponse	$T=10, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	180
RequestResponse	$T=10, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	180
RequestResponse	$T=10, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	180
RequestResponse	$T=10, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	180
RequestResponse	$T=10, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	180
RequestResponse	$T=10, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	180
RequestResponse	$T=10, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	180
RequestResponse	$T=10, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	180
RequestResponse	$T=10, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	180
RequestResponse	$T=30, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	60
RequestResponse	$T=30, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	60
RequestResponse	$T=30, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	60
RequestResponse	$T=30, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	60
RequestResponse	$T=30, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	60
RequestResponse	$T=30, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	60
RequestResponse	$T=30, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	60

Figure 4.3a. Requests sent by monitoringApp.app[0], the first of four pollers. This upper portion shows the 1 s, 10 s and part of the 30 s cases across both background loads.



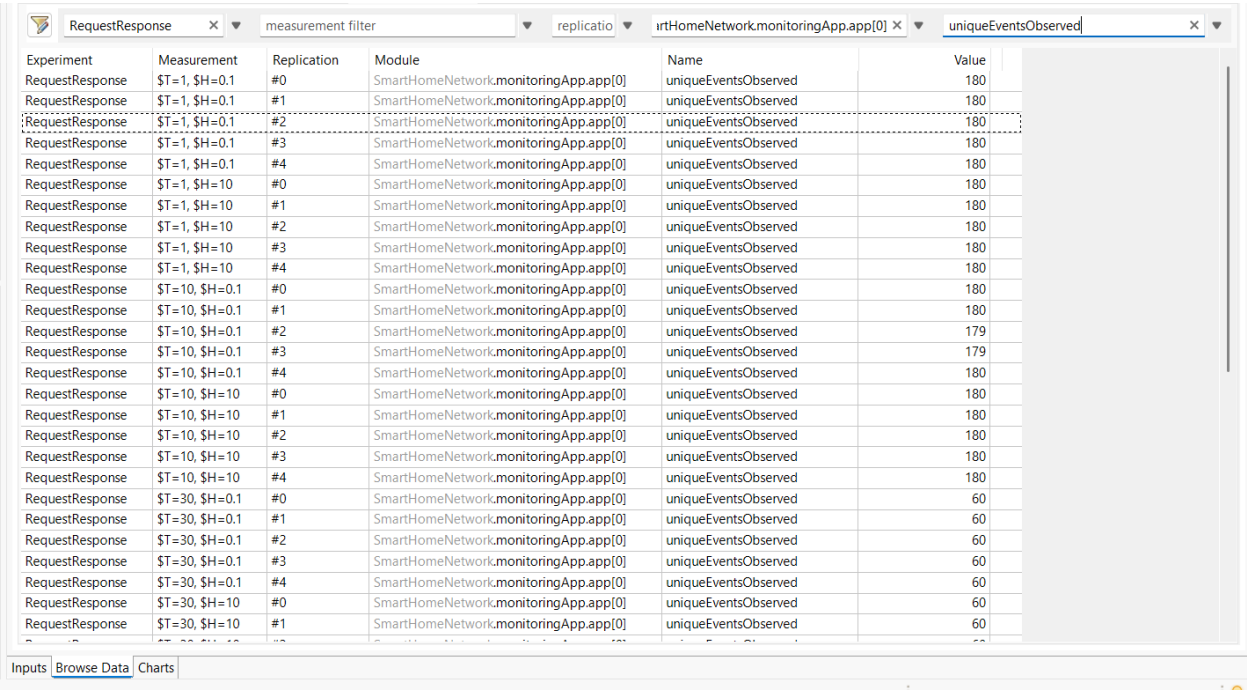
RequestResponse | measurement filter | replicatio | ırtHomeNetwork.monitoringApp.app[0] | packetSent:count

Experiment	Measurement	Replication	Module	Name	Value
RequestResponse	$T=30, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	60
RequestResponse	$T=30, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	60
RequestResponse	$T=30, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	60
RequestResponse	$T=30, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	60
RequestResponse	$T=30, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	60
RequestResponse	$T=30, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	60
RequestResponse	$T=30, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	60
RequestResponse	$T=5, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	360
RequestResponse	$T=5, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	360
RequestResponse	$T=5, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	360
RequestResponse	$T=5, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	360
RequestResponse	$T=5, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	360
RequestResponse	$T=5, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	360
RequestResponse	$T=5, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	360
RequestResponse	$T=5, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	360
RequestResponse	$T=5, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	360
RequestResponse	$T=5, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	360
RequestResponse	$T=60, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	30
RequestResponse	$T=60, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	30
RequestResponse	$T=60, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	30
RequestResponse	$T=60, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	30
RequestResponse	$T=60, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	30
RequestResponse	$T=60, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	30
RequestResponse	$T=60, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	30
RequestResponse	$T=60, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	30
RequestResponse	$T=60, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	30
RequestResponse	$T=60, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	packetSent:count	30

Inputs | Browse Data | Charts

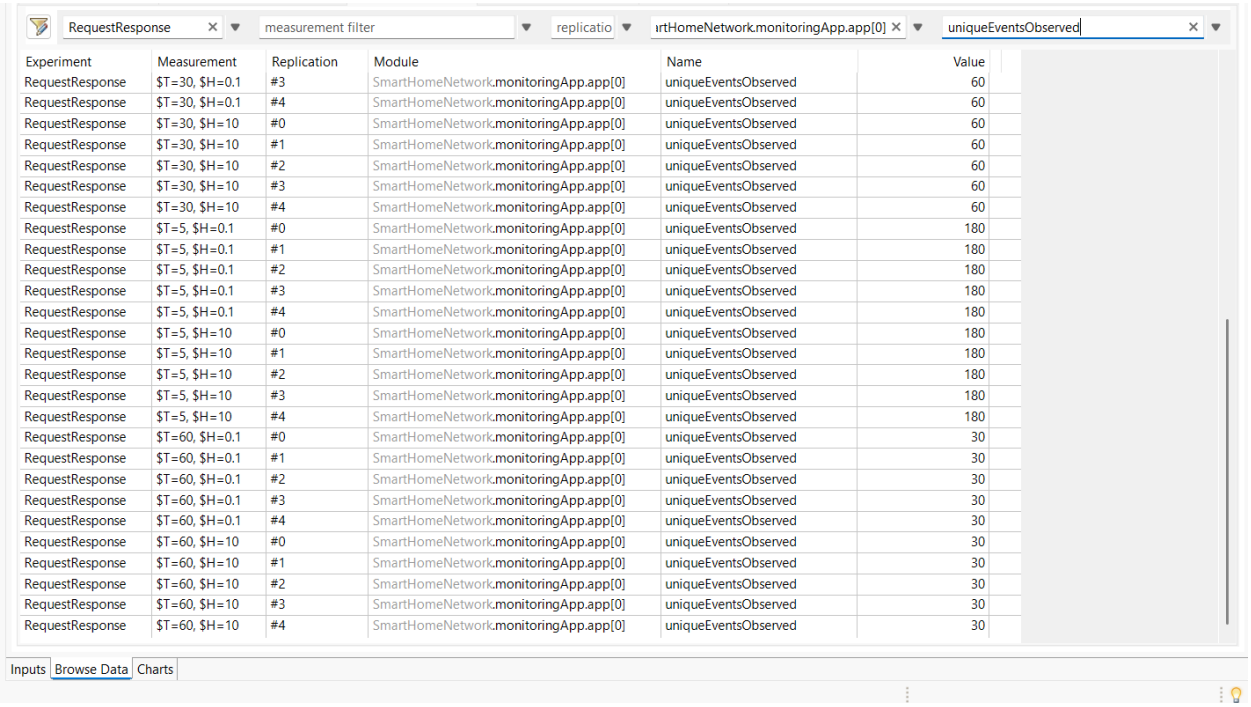
Figure 4.3b. Continuation of the same request-count table. The 5 s and 60 s cases send 360 and 30 requests per run, respectively. Overlapping rows retain the context of the scrolled view; they are not additional runs.

Experiment 4 - Polling Coverage Screenshots



Experiment	Measurement	Replication	Module	Name	Value
RequestResponse	$T=1, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=1, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=1, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=1, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=1, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=1, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=1, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=1, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=1, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=1, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=10, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=10, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=10, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	179
RequestResponse	$T=10, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	179
RequestResponse	$T=10, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=10, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=10, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=10, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=10, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=10, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=30, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	60
RequestResponse	$T=30, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	60
RequestResponse	$T=30, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	60
RequestResponse	$T=30, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	60
RequestResponse	$T=30, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	60
RequestResponse	$T=30, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	60
RequestResponse	$T=30, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	60

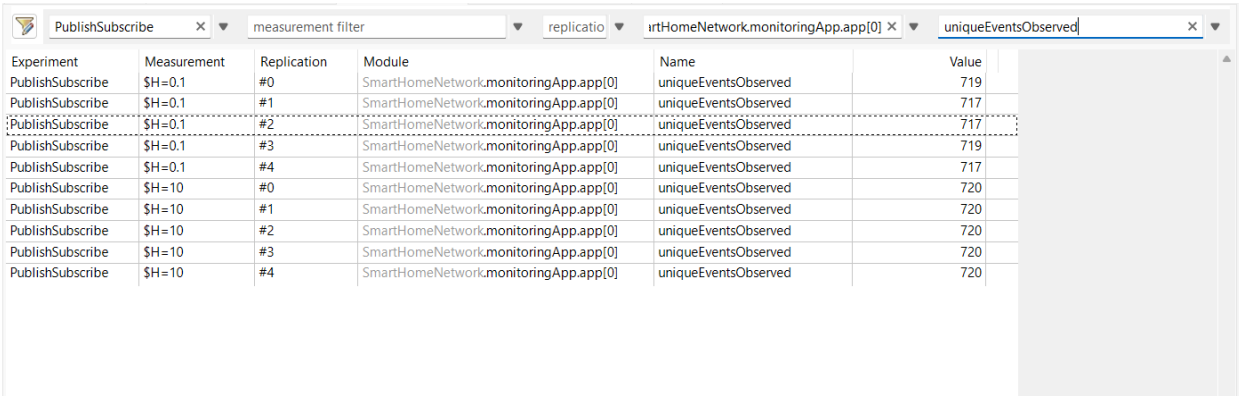
Figure 4.4a. Distinct events observed by the first poller. Its sensor generates 180 updates per run. At T = 10 s and high background load (H = 0.1 s), replications #2 and #3 each observe 179 of those updates.



Experiment	Measurement	Replication	Module	Name	Value
RequestResponse	$T=30, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	60
RequestResponse	$T=30, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	60
RequestResponse	$T=30, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	60
RequestResponse	$T=30, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	60
RequestResponse	$T=30, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	60
RequestResponse	$T=30, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	60
RequestResponse	$T=30, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	60
RequestResponse	$T=5, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=5, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=5, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=5, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=5, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=5, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=5, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=5, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=5, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=5, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	180
RequestResponse	$T=60, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	30
RequestResponse	$T=60, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	30
RequestResponse	$T=60, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	30
RequestResponse	$T=60, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	30
RequestResponse	$T=60, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	30
RequestResponse	$T=60, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	30
RequestResponse	$T=60, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	30
RequestResponse	$T=60, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	30
RequestResponse	$T=60, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	30
RequestResponse	$T=60, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	30

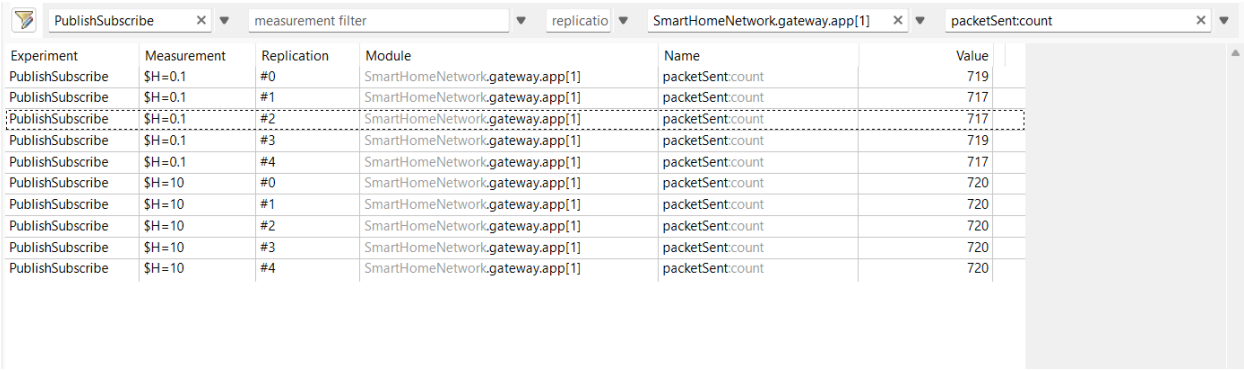
Figure 4.4b. Continuation of the event-coverage table. The first poller observes 60 events at T = 30 s and 30 at T = 60 s. These screenshots show one sensor; Table 4.1 combines the four sensors and five repetitions.

Experiment 4 - Subscriber and Broker Screenshots



Experiment	Measurement	Replication	Module	Name	Value
PublishSubscribe	$H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	719
PublishSubscribe	$H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	717
PublishSubscribe	$H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	717
PublishSubscribe	$H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	719
PublishSubscribe	$H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	717
PublishSubscribe	$H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	720
PublishSubscribe	$H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	720
PublishSubscribe	$H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	720
PublishSubscribe	$H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	720
PublishSubscribe	$H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	uniqueEventsObserved	720

Figure 4.5. Unique events received by the single core subscriber from all four sensors. Normal load delivers all 720 events per run; high load delivers 717-719. H is the background interval, so H = 0.1 s represents the higher traffic load.

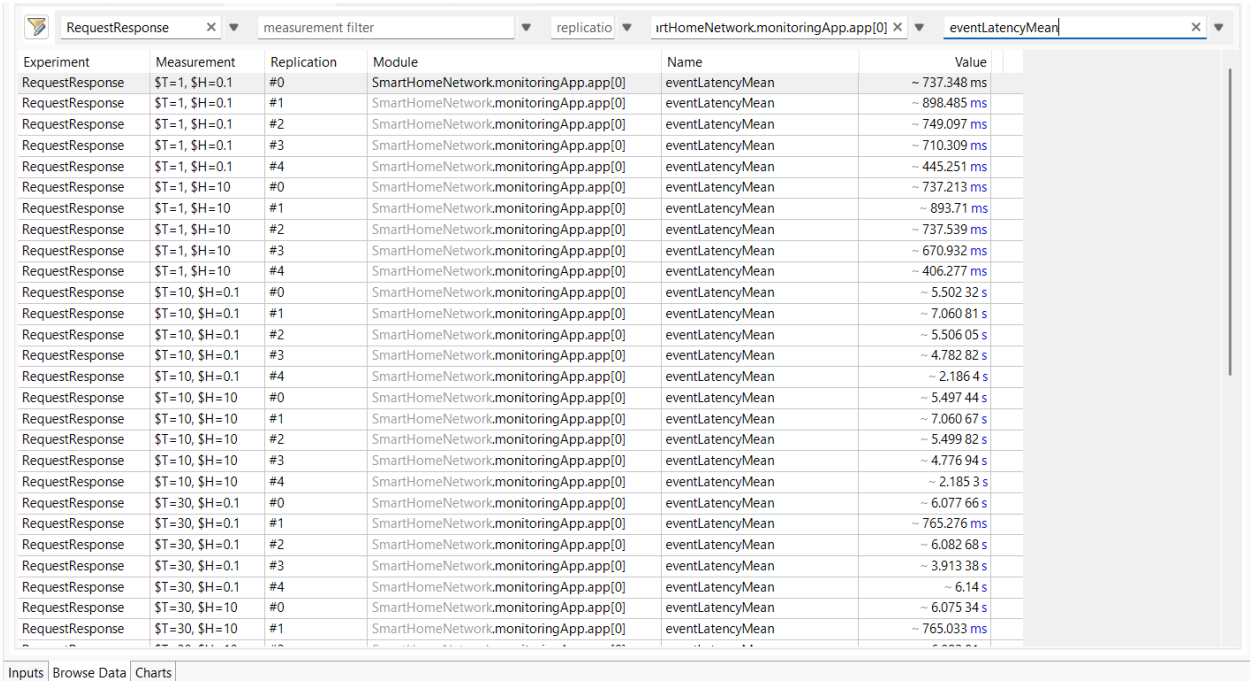


Experiment	Measurement	Replication	Module	Name	Value
PublishSubscribe	$H=0.1	#0	SmartHomeNetwork.gateway.app[1]	packetSent:count	719
PublishSubscribe	$H=0.1	#1	SmartHomeNetwork.gateway.app[1]	packetSent:count	717
PublishSubscribe	$H=0.1	#2	SmartHomeNetwork.gateway.app[1]	packetSent:count	717
PublishSubscribe	$H=0.1	#3	SmartHomeNetwork.gateway.app[1]	packetSent:count	719
PublishSubscribe	$H=0.1	#4	SmartHomeNetwork.gateway.app[1]	packetSent:count	717
PublishSubscribe	$H=10	#0	SmartHomeNetwork.gateway.app[1]	packetSent:count	720
PublishSubscribe	$H=10	#1	SmartHomeNetwork.gateway.app[1]	packetSent:count	720
PublishSubscribe	$H=10	#2	SmartHomeNetwork.gateway.app[1]	packetSent:count	720
PublishSubscribe	$H=10	#3	SmartHomeNetwork.gateway.app[1]	packetSent:count	720
PublishSubscribe	$H=10	#4	SmartHomeNetwork.gateway.app[1]	packetSent:count	720

Figure 4.6. Notifications sent by gateway.app[1], the broker. Counts match the subscriber observations in Figure 4.5 for every run, consistent with no further notification loss on the broker-to-core leg in these runs.

The screenshots document packet and event counts. The verified event-latency measurements are presented in Table 4.1 and Figure 4.1.

Experiment 4 - Mean Polling Latency Screenshots

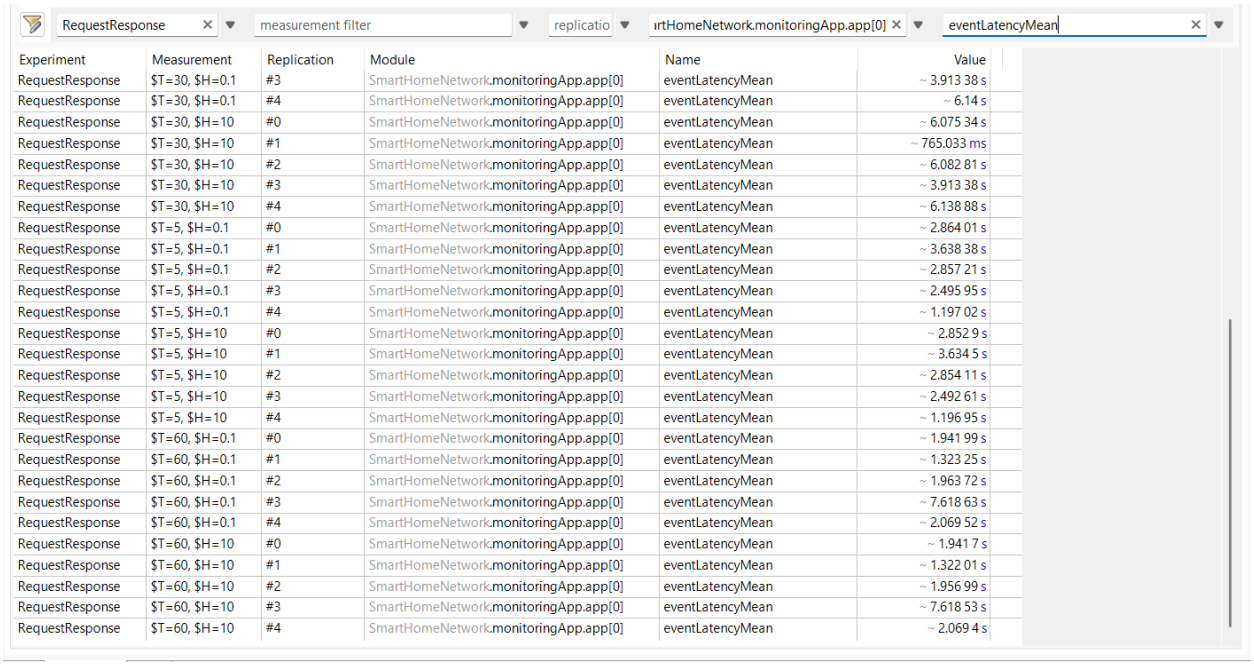


RequestResponse | measurement filter | replicatio | ırtHomeNetwork.monitoringApp.app[0] | eventLatencyMean

Experiment	Measurement	Replication	Module	Name	Value
RequestResponse	$T=1, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 737.348 ms
RequestResponse	$T=1, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 898.485 ms
RequestResponse	$T=1, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 749.097 ms
RequestResponse	$T=1, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 710.309 ms
RequestResponse	$T=1, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 445.251 ms
RequestResponse	$T=1, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 737.213 ms
RequestResponse	$T=1, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 893.71 ms
RequestResponse	$T=1, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 737.539 ms
RequestResponse	$T=1, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 670.932 ms
RequestResponse	$T=1, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 406.277 ms
RequestResponse	$T=10, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 5.502 32 s
RequestResponse	$T=10, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 7.060 81 s
RequestResponse	$T=10, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 5.506 05 s
RequestResponse	$T=10, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 4.782 82 s
RequestResponse	$T=10, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 2.186 4 s
RequestResponse	$T=10, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 5.497 44 s
RequestResponse	$T=10, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 7.060 67 s
RequestResponse	$T=10, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 5.499 82 s
RequestResponse	$T=10, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 4.776 94 s
RequestResponse	$T=10, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 2.185 3 s
RequestResponse	$T=30, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 6.077 66 s
RequestResponse	$T=30, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 765.276 ms
RequestResponse	$T=30, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 6.082 68 s
RequestResponse	$T=30, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 3.913 38 s
RequestResponse	$T=30, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 6.14 s
RequestResponse	$T=30, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 6.075 34 s
RequestResponse	$T=30, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 765.033 ms

Inputs | Browse Data | Charts

Figure 4.7a. Mean event-to-core latency for monitoringApp.app[0], the first poller. Values include waiting for a poll, not just request/reply transport time. The IDE displays either milliseconds or seconds (1 s = 1000 ms).

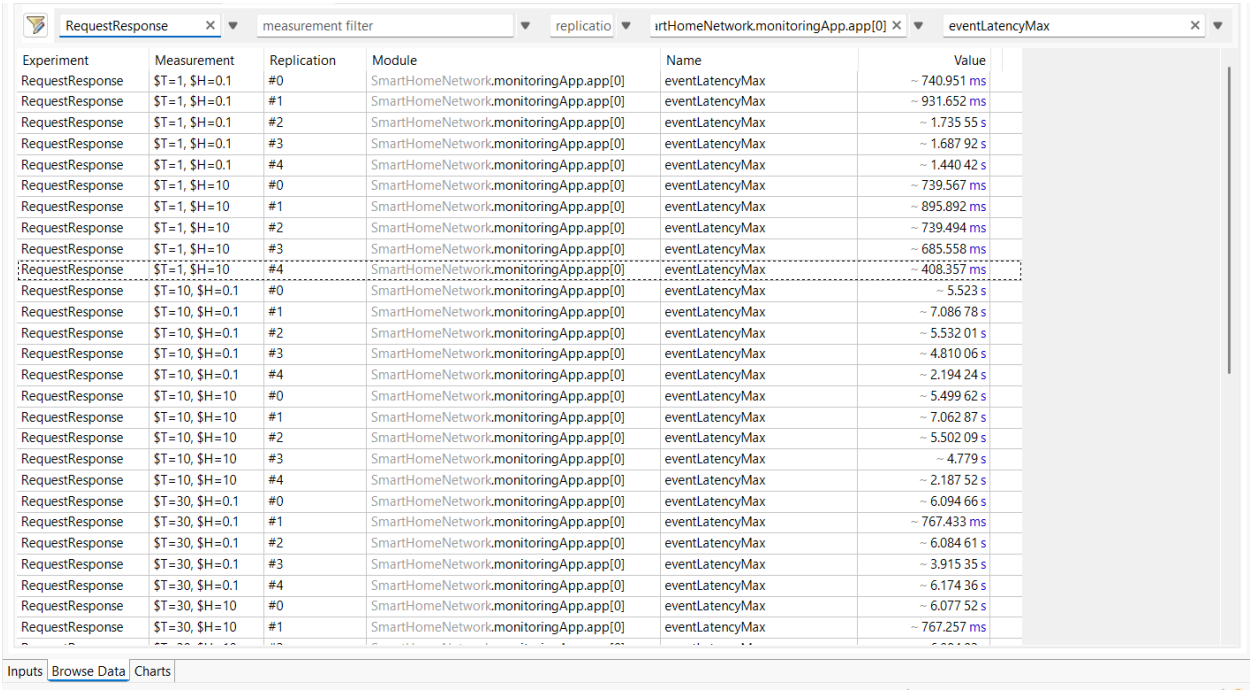


RequestResponse | measurement filter | replicatio | ırtHomeNetwork.monitoringApp.app[0] | eventLatencyMean

Experiment	Measurement	Replication	Module	Name	Value
RequestResponse	$T=30, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 3.913 38 s
RequestResponse	$T=30, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 6.14 s
RequestResponse	$T=30, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 6.075 34 s
RequestResponse	$T=30, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 765.033 ms
RequestResponse	$T=30, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 6.082 81 s
RequestResponse	$T=30, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 3.913 38 s
RequestResponse	$T=30, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 6.138 88 s
RequestResponse	$T=5, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 2.864 01 s
RequestResponse	$T=5, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 3.638 38 s
RequestResponse	$T=5, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 2.857 21 s
RequestResponse	$T=5, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 2.495 95 s
RequestResponse	$T=5, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 1.197 02 s
RequestResponse	$T=5, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 2.852 9 s
RequestResponse	$T=5, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 3.634 5 s
RequestResponse	$T=5, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 2.854 11 s
RequestResponse	$T=5, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 2.492 61 s
RequestResponse	$T=5, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 1.196 95 s
RequestResponse	$T=60, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 1.941 99 s
RequestResponse	$T=60, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 1.323 25 s
RequestResponse	$T=60, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 1.963 72 s
RequestResponse	$T=60, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 7.618 63 s
RequestResponse	$T=60, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 2.069 52 s
RequestResponse	$T=60, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 1.941 7 s
RequestResponse	$T=60, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 1.322 01 s
RequestResponse	$T=60, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 1.956 99 s
RequestResponse	$T=60, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 7.618 53 s
RequestResponse	$T=60, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 2.069 4 s

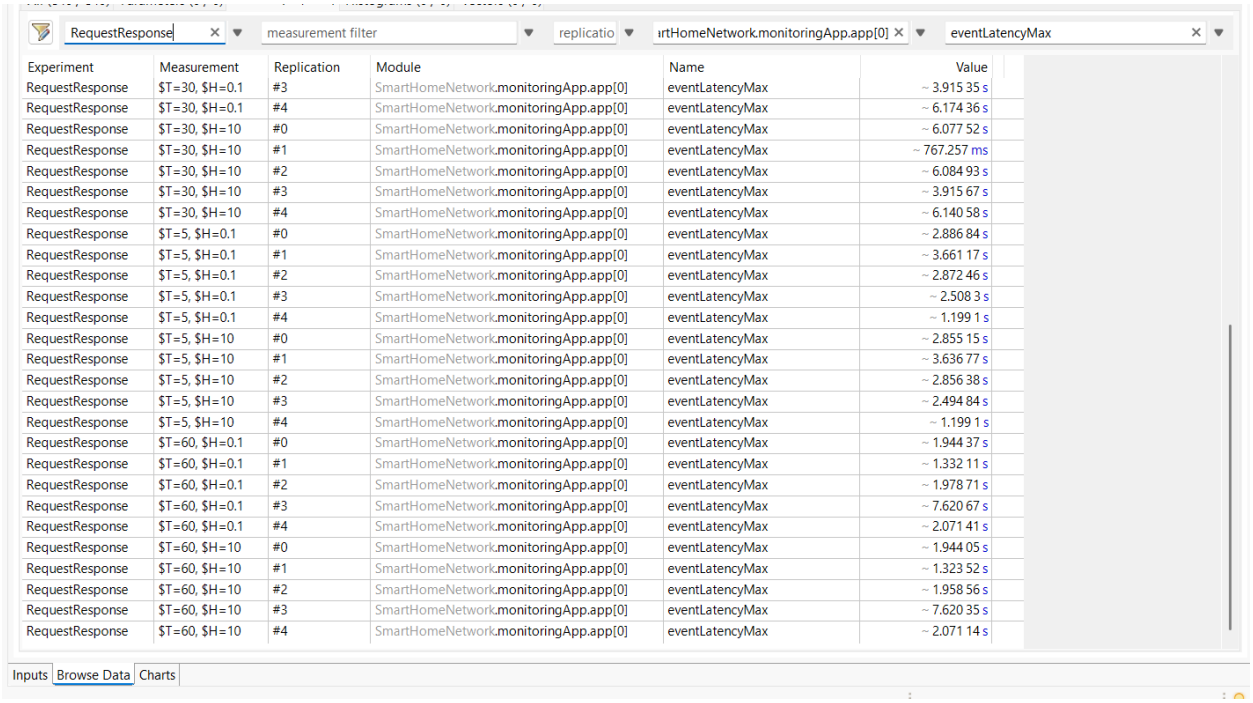
Figure 4.7b. Continuation of the first poller's mean latency table. These are per-run means for one sensor; Table 4.1 combines all four pollers within each run and then averages five repetitions. Slow-polling latency excludes events never observed, so it must be read with coverage.

Experiment 4 - Maximum Polling Latency Screenshots



Experiment	Measurement	Replication	Module	Name	Value
RequestResponse	$T=1, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 740.951 ms
RequestResponse	$T=1, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 931.652 ms
RequestResponse	$T=1, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 1.735 55 s
RequestResponse	$T=1, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 1.687 92 s
RequestResponse	$T=1, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 1.440 42 s
RequestResponse	$T=1, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 739.567 ms
RequestResponse	$T=1, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 895.892 ms
RequestResponse	$T=1, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 739.494 ms
RequestResponse	$T=1, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 685.558 ms
RequestResponse	$T=1, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 408.357 ms
RequestResponse	$T=10, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 5.523 s
RequestResponse	$T=10, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 7.086 78 s
RequestResponse	$T=10, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 5.532 01 s
RequestResponse	$T=10, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 4.810 06 s
RequestResponse	$T=10, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 2.194 24 s
RequestResponse	$T=10, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 5.499 62 s
RequestResponse	$T=10, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 7.062 87 s
RequestResponse	$T=10, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 5.502 09 s
RequestResponse	$T=10, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 4.779 s
RequestResponse	$T=10, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 2.187 52 s
RequestResponse	$T=30, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 6.094 66 s
RequestResponse	$T=30, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 767.433 ms
RequestResponse	$T=30, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 6.084 61 s
RequestResponse	$T=30, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 3.915 35 s
RequestResponse	$T=30, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 6.174 36 s
RequestResponse	$T=30, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 6.077 52 s
RequestResponse	$T=30, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 767.257 ms

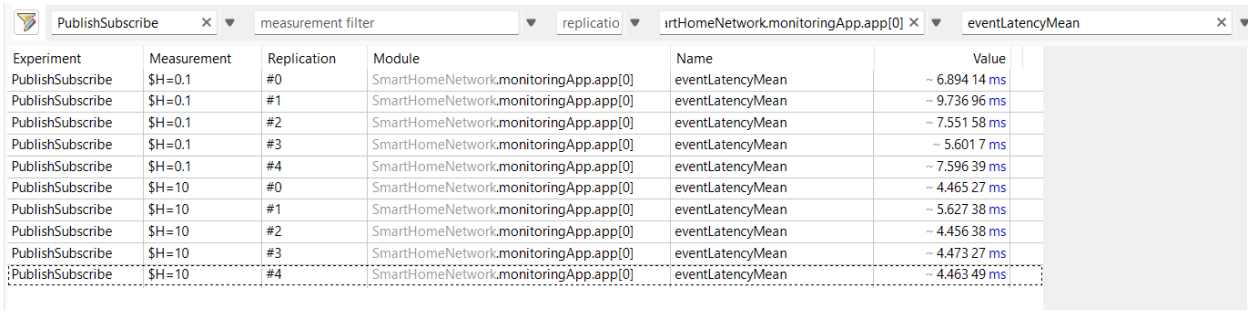
Figure 4.8a. Maximum observed event latency for the first poller, showing T = 1 s, 10 s and part of 30 s. These are exact statistic maxima displayed with rounded values, not histogram range boundaries.



Experiment	Measurement	Replication	Module	Name	Value
RequestResponse	$T=30, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 3.915 35 s
RequestResponse	$T=30, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 6.174 36 s
RequestResponse	$T=30, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 6.077 52 s
RequestResponse	$T=30, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 767.257 ms
RequestResponse	$T=30, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 6.084 93 s
RequestResponse	$T=30, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 3.915 67 s
RequestResponse	$T=30, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 6.140 58 s
RequestResponse	$T=5, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 2.886 84 s
RequestResponse	$T=5, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 3.661 17 s
RequestResponse	$T=5, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 2.872 46 s
RequestResponse	$T=5, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 2.508 3 s
RequestResponse	$T=5, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 1.199 1 s
RequestResponse	$T=5, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 2.855 15 s
RequestResponse	$T=5, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 3.636 77 s
RequestResponse	$T=5, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 2.856 38 s
RequestResponse	$T=5, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 2.494 84 s
RequestResponse	$T=5, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 1.199 1 s
RequestResponse	$T=60, $H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 1.944 37 s
RequestResponse	$T=60, $H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 1.332 11 s
RequestResponse	$T=60, $H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 1.978 71 s
RequestResponse	$T=60, $H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 7.620 67 s
RequestResponse	$T=60, $H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 2.071 41 s
RequestResponse	$T=60, $H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 1.944 05 s
RequestResponse	$T=60, $H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 1.323 52 s
RequestResponse	$T=60, $H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 1.958 56 s
RequestResponse	$T=60, $H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 7.620 35 s
RequestResponse	$T=60, $H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 2.071 14 s

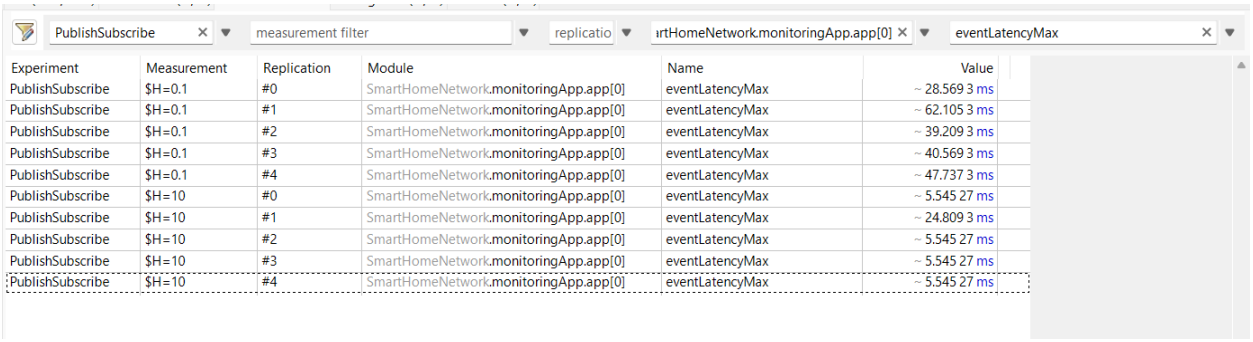
Figure 4.8b. Continuation of the maximum latency table. Overlapping rows are repeated screen context, not extra runs. Table 4.1 reports the largest maximum over all four pollers and five seeds, so it can exceed the individual values shown here.

Experiment 4 - Publish/Subscribe Latency Screenshots



Experiment	Measurement	Replication	Module	Name	Value
PublishSubscribe	$H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 6.894 14 ms
PublishSubscribe	$H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 9.736 96 ms
PublishSubscribe	$H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 7.551 58 ms
PublishSubscribe	$H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 5.601 7 ms
PublishSubscribe	$H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 7.596 39 ms
PublishSubscribe	$H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 4.465 27 ms
PublishSubscribe	$H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 5.627 38 ms
PublishSubscribe	$H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 4.456 38 ms
PublishSubscribe	$H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 4.473 27 ms
PublishSubscribe	$H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMean	~ 4.463 49 ms

Figure 4.9. Mean sensor-event-to-core latency at the single subscriber. Each row combines all received events from the four publishers in one run. Averaging the five per-run means gives 4.70 ms at normal load and 7.48 ms at high load.



Experiment	Measurement	Replication	Module	Name	Value
PublishSubscribe	$H=0.1	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 28.569 3 ms
PublishSubscribe	$H=0.1	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 62.105 3 ms
PublishSubscribe	$H=0.1	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 39.209 3 ms
PublishSubscribe	$H=0.1	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 40.569 3 ms
PublishSubscribe	$H=0.1	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 47.737 3 ms
PublishSubscribe	$H=10	#0	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 5.545 27 ms
PublishSubscribe	$H=10	#1	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 24.809 3 ms
PublishSubscribe	$H=10	#2	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 5.545 27 ms
PublishSubscribe	$H=10	#3	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 5.545 27 ms
PublishSubscribe	$H=10	#4	SmartHomeNetwork.monitoringApp.app[0]	eventLatencyMax	~ 5.545 27 ms

Figure 4.10. Maximum event-to-core latency per publishing run. The worst observed values across five seeds are 24.81 ms at normal load (H = 10 s) and 62.11 ms at high load (H = 0.1 s).

Figures 4.7-4.10 use a derived scalar view of the original eventLatency:stats records: eventLatencyMean copies the mean field and eventLatencyMax copies the max field. The displayed values are the existing measurements, not newly simulated results. Polling and publishing both measure from the original sensor value change to its first observation at the core.